USQL Language Reference

USQL is UQA's SQL dialect — a PostgreSQL-compatible SQL language extended with cross-paradigm search, graph traversal, and probabilistic fusion operators.

USQL Language Reference

- 1. Data Types
  - Numeric Types
  - Text Types
  - Boolean
  - Date/Time Types
  - JSON Types
  - Special Types
- 2. Data Definition Language (DDL)
  - CREATE TABLE
  - ALTER TABLE
  - DROP TABLE
  - TRUNCATE
  - Indexes
  - Views
  - Sequences
  - Foreign Data Wrappers
- 3. Data Manipulation Language (DML)
  - INSERT
  - UPDATE
  - DELETE
- 4. Queries (SELECT)
  - Basic SELECT
  - DISTINCT
  - Column Expressions
  - WHERE Clause
  - GROUP BY
  - HAVING
  - ORDER BY
  - LIMIT / OFFSET
  - Set Operations
- 5. JOINs
  - INNER JOIN
  - LEFT / RIGHT / FULL OUTER JOIN
  - CROSS JOIN
  - LATERAL JOIN
  - Multi-Way Joins
- 6. Subqueries
  - Scalar Subquery
  - IN Subquery
  - EXISTS Subquery
  - Derived Table (Subquery in FROM)
- 7. Common Table Expressions (CTEs)
  - Simple CTE
  - Multiple CTEs
  - Recursive CTE
- 8. Window Functions
  - Syntax

- Ranking Functions
- Value Functions
- Aggregate Window Functions
- Window Frame Specification
- Named Windows

9. Aggregate Functions
    - Basic Aggregates
    - Collection Aggregates
    - Statistical Aggregates
    - Regression Aggregates
    - FILTER Clause
    - Ordered Aggregates
10. Built-in Scalar Functions
    - String Functions
    - Regular Expression Functions
    - Math Functions
    - Trigonometric Functions
    - Date/Time Functions
    - Conditional Functions
    - CASE Expression
    - Type Conversion
    - Sequence Functions
    - Spatial Functions
    - Miscellaneous
11. JSON/JSONB Operators and Functions
    - Access Operators
    - Containment Operators
    - Construction Functions
    - Inspection Functions
    - Modification Functions
    - Set-Returning Functions
12. Array Functions
13. Cross-Paradigm Search Functions
    - Full-Text Search
    - Vector Similarity Search
    - Graph Traversal
    - Bounded RPQ
    - Graph Centrality
    - Weighted Path Query
    - Graph Mutation (SQL)
    - Progressive Fusion
    - Spatial Search
    - Vector Exclusion
14. Probabilistic Fusion
    - Log-Odds Fusion
    - Log-Odds Fusion with Gating
    - Probabilistic AND
    - Probabilistic OR
    - Probabilistic NOT
    - Spatial + Fusion
15. Cypher / Graph Integration
    - Named Graph Management
    - Cypher Queries in SQL
    - Supported Cypher Clauses
    - Cypher Pattern Syntax

Cypher Functions
16. Table-Returning Functions
GENERATE_SERIES
REGEXP_SPLIT_TO_TABLE
17. Information Schema
Tables Metadata
Column Metadata
18. EXPLAIN
19. ANALYZE
20. Transactions
21. Prepared Statements
22. Operators Reference
Comparison
Arithmetic
String
Pattern Matching
Logical
Range / Set
JSON

---

1. Data Types

Numeric Types

Type	Aliases	Description
`INTEGER`	`INT`, `INT4`	64-bit signed integer
`SMALLINT`	`INT2`	16-bit signed integer
`BIGINT`	`INT8`	64-bit signed integer
`SERIAL`		Auto-incrementing integer
`BIGSERIAL`		Auto-incrementing 64-bit integer
`REAL`	`FLOAT4`	Single-precision floating point
`DOUBLE PRECISION`	`FLOAT`, `FLOAT8`	Double-precision floating point
`NUMERIC(p,s)`	`DECIMAL(p,s)`	Fixed-point with precision and scale

Text Types

Type	Description
`TEXT`	Variable-length string (unlimited)
`VARCHAR(n)`	Variable-length string (max n characters)
`CHAR(n)`	Fixed-length string (n characters)
`NAME`	Internal identifier type

Boolean

Type	Description
`BOOLEAN`	`TRUE` / `FALSE` / `NULL`

Date/Time Types

Type	Description
`DATE`	Calendar date (year, month, day)
`TIMESTAMP`	Date and time without timezone
`TIMESTAMPTZ`	Date and time with timezone
`INTERVAL`	Time duration

JSON Types

Type	Description
`JSON`	JSON text stored as-is
`JSONB`	Binary JSON with operator support

Special Types

Type	Description
`UUID`	Universally unique identifier
`BYTEA`	Binary data
`VECTOR(n)`	Fixed-dimension vector for similarity search
`POINT`	2-D coordinate (longitude, latitude) for spatial queries
`type[]`	Array of any base type (e.g. `INTEGER[]`, `TEXT[]`)

---

2. Data Definition Language (DDL)

CREATE TABLE

```sql
CREATE TABLE table_name (
    column_name data_type [column_constraints],
    ...
    [table_constraints]
);

-- Create from query
CREATE TABLE new_table AS SELECT ... ;
```

Column constraints:

```sql
col_name type PRIMARY KEY
col_name type NOT NULL
col_name type UNIQUE
col_name type DEFAULT value
col_name type CHECK (expression)
col_name type REFERENCES parent_table(parent_col)
```

Composite constraints:

```sql
CREATE TABLE t (
    a INTEGER,
    b INTEGER,
    PRIMARY KEY (a, b),
    UNIQUE (a, b),
    CHECK (a > 0),
    FOREIGN KEY (a) REFERENCES other(id)
);
```

ALTER TABLE

```sql
ALTER TABLE name ADD COLUMN col_name type [DEFAULT value];
ALTER TABLE name DROP COLUMN [IF EXISTS] col_name;
ALTER TABLE name RENAME COLUMN old_name TO new_name;
ALTER TABLE name RENAME TO new_name;
ALTER TABLE name ALTER COLUMN col_name SET DEFAULT value;
ALTER TABLE name ALTER COLUMN col_name DROP DEFAULT;
ALTER TABLE name ALTER COLUMN col_name SET NOT NULL;
ALTER TABLE name ALTER COLUMN col_name DROP NOT NULL;
```

DROP TABLE

```sql
DROP TABLE [IF EXISTS] table_name [, ...];
```

TRUNCATE

```sql
TRUNCATE [TABLE] table_name [RESTART IDENTITY];
```

Indexes

```sql
-- B-tree index (default)
CREATE [UNIQUE] INDEX index_name ON table_name (col1 [ASC|DESC], ...);

-- HNSW vector index
CREATE INDEX index_name ON table_name USING hnsw (vector_col);
CREATE INDEX index_name ON table_name USING hnsw (vector_col)
    WITH (ef_construction = 200, m = 16);

-- R*Tree spatial index
CREATE INDEX index_name ON table_name USING rtree (point_col);

DROP INDEX [IF EXISTS] index_name;
```

The `USING hnsw` clause creates an HNSW (Hierarchical Navigable Small World) index on a `VECTOR(n)` column for approximate nearest neighbor search. Optional `WITH` parameters control index quality: `ef_construction` (default 200) sets build-time search depth, and `m` (default 16) sets the number of bi-directional links per node.

The `USING rtree` clause creates an R*Tree spatial index on a `POINT` column for O(log N) spatial range queries. The RTree stores each point as a degenerate bounding box in SQLite's built-in R*Tree virtual table module. Without an index, `spatial_within()` falls back to brute-force Haversine scan.

Views

```sql
CREATE VIEW view_name AS SELECT ...;
DROP VIEW [IF EXISTS] view_name;
```

Sequences

```sql
CREATE SEQUENCE seq_name [START WITH n] [INCREMENT BY n] [CYCLE|NO CYCLE];
ALTER SEQUENCE seq_name [START WITH n] [INCREMENT BY n];
DROP SEQUENCE [IF EXISTS] seq_name;
```

Foreign Data Wrappers

Foreign Data Wrappers (FDW) let you query external data sources -- Parquet files, CSV files, S3 buckets, attached DuckDB databases, or remote Arrow Flight SQL servers -- as if they were regular tables.

DDL Syntax:

```sql
-- Create a server backed by DuckDB (in-process analytics engine)
CREATE SERVER name FOREIGN DATA WRAPPER duckdb_fdw;

-- Create a server backed by Arrow Flight SQL (remote query engine)
CREATE SERVER name FOREIGN DATA WRAPPER arrow_fdw
    OPTIONS (host 'hostname', port '8815');

```

```sql
-- Create a foreign table pointing to a Parquet file
CREATE FOREIGN TABLE name (col_name type, ...)
    SERVER server_name OPTIONS (source '/path/to/file.parquet');

-- Create a foreign table pointing to a CSV file
CREATE FOREIGN TABLE name (col_name type, ...)
    SERVER server_name OPTIONS (source '/path/to/file.csv');

-- Explicit DuckDB reader expression (e.g., Hive-partitioned directory)
CREATE FOREIGN TABLE name (col_name type, ...)
    SERVER server_name
    OPTIONS (source '/data/events/**/*.parquet',
             hive_partitioning 'true');

DROP FOREIGN SERVER [IF EXISTS] name;
DROP FOREIGN TABLE [IF EXISTS] name;
```

Foreign tables are read-only. INSERT, UPDATE, and DELETE are rejected.

Full Query Pushdown:

When every table referenced in a SELECT lives on the same DuckDB server, UQA pushes the entire query -- including JOINs, GROUP BY, ORDER BY, LIMIT, window functions, and subqueries -- down to DuckDB for execution. No rows are materialized in Python; DuckDB processes the data natively, making large-scale analytics fast even on datasets with tens of millions of rows.

The pushdown path works as follows:

1. UQA walks the AST to collect all table references (FROM, JOINs, subqueries).
2. If all tables resolve to foreign tables on the same DuckDB server, each foreign table is registered as a DuckDB view backed by its source (e.g., `read_parquet(...)` or `read_csv_auto(...)`).
3. The original SQL is deparsed from the AST and executed directly by DuckDB.
4. The Arrow result is converted back to a USQL result set.

```sql
-- Setup: Parquet file with 41M NYC taxi rows
CREATE SERVER analytics FOREIGN DATA WRAPPER duckdb_fdw;
CREATE FOREIGN TABLE nyc_taxi (
    pickup_zone TEXT,
    dropoff_zone TEXT,
    fare DOUBLE PRECISION,
    trip_distance DOUBLE PRECISION,
    passenger_count INTEGER
) SERVER analytics OPTIONS (source '/data/nyc_taxi_2023.parquet');

-- This entire query executes inside DuckDB -- no row-by-row Python overhead
SELECT pickup_zone,
       COUNT(*) AS num_trips,
       ROUND(AVG(fare), 2) AS avg_fare,
       ROUND(AVG(trip_distance), 2) AS avg_distance
FROM nyc_taxi
WHERE passenger_count >= 1
GROUP BY pickup_zone
ORDER BY num_trips DESC
```

```
LIMIT 10;
```

Mixed Foreign-Local JOINs:

When a query joins foreign tables with local UQA tables on the same DuckDB server, UQA ships the local table data to DuckDB as a temporary table and executes the full query in DuckDB. This avoids row-by-row Python hash joins and lets DuckDB handle the heavy lifting. Local tables up to 100,000 rows are eligible for shipping.

```
-- Local table with curated metadata
CREATE TABLE zone_metadata (
    zone_name TEXT PRIMARY KEY,
    borough TEXT NOT NULL,
    is_airport BOOLEAN DEFAULT FALSE
);
INSERT INTO zone_metadata (zone_name, borough, is_airport) VALUES
    ('JFK Airport', 'Queens', TRUE),
    ('LaGuardia Airport', 'Queens', TRUE),
    ('Times Sq/Theatre District', 'Manhattan', FALSE);

-- Mixed join: local zone_metadata is shipped to DuckDB, then the entire
-- query -- including the JOIN, GROUP BY, and ORDER BY -- runs in DuckDB
SELECT z.borough,
       z.is_airport,
       COUNT(*) AS num_trips,
       ROUND(AVG(t.fare), 2) AS avg_fare
FROM nyc_taxi t
INNER JOIN zone_metadata z ON t.pickup_zone = z.zone_name
GROUP BY z.borough, z.is_airport
ORDER BY num_trips DESC;
```

Multiple Foreign Tables on the Same Server:

JOINs between two or more foreign tables on the same server are also pushed down entirely to DuckDB:

```
CREATE FOREIGN TABLE orders (
    order_id INTEGER, product_id INTEGER,
    customer TEXT, quantity INTEGER
) SERVER analytics OPTIONS (source '/data/orders.parquet');

CREATE FOREIGN TABLE products (
    product_id INTEGER, name TEXT,
    category TEXT, price DOUBLE PRECISION
) SERVER analytics OPTIONS (source '/data/products.parquet');

-- Pushed down to DuckDB as a single query
SELECT o.customer,
       COUNT(*) AS num_orders,
       SUM(p.price * o.quantity) AS total_spent
FROM orders o
INNER JOIN products p ON o.product_id = p.product_id
GROUP BY o.customer
ORDER BY total_spent DESC
```

```
LIMIT 10;
```

Predicate Pushdown for Single-Table Scans:

Even when full query pushdown does not apply (e.g., the query mixes search operators like `text_match()` with foreign tables), UQA extracts pushable WHERE predicates and forwards them to the FDW handler. Comparison operators (`=`, `<`, `>`, `<=`, `>=`, `!=`), `LIKE`, `ILIKE`, `BETWEEN`, `IN`, and `IS NULL` / `IS NOT NULL` are pushed down. Non-pushable predicates (function calls, cross-table references) are evaluated after the scan.

---

3. Data Manipulation Language (DML)

INSERT

```
-- Single row
INSERT INTO table (col1, col2) VALUES (val1, val2);

-- Multiple rows
INSERT INTO table (col1, col2) VALUES (1, 'a'), (2, 'b'), (3, 'c');

-- Insert from SELECT
INSERT INTO table (col1, col2) SELECT ... ;

-- ON CONFLICT (upsert)
INSERT INTO table (id, name, count) VALUES (1, 'x', 1)
    ON CONFLICT (id) DO NOTHING;

INSERT INTO table (id, name, count) VALUES (1, 'x', 1)
    ON CONFLICT (id) DO UPDATE SET count = EXCLUDED.count;

-- RETURNING
INSERT INTO table (name) VALUES ('Alice') RETURNING id, name;
INSERT INTO table (name) VALUES ('Alice') RETURNING *;
```

The `EXCLUDED` pseudo-table refers to the row that was proposed for insertion.

UPDATE

```
UPDATE table SET col1 = val1, col2 = val2 WHERE condition;

-- UPDATE with FROM
UPDATE table SET col1 = t2.val FROM other_table t2 WHERE table.id = t2.id;

-- RETURNING
UPDATE table SET name = 'Bob' WHERE id = 1 RETURNING *;
```

DELETE

```sql
DELETE FROM table WHERE condition;

-- DELETE with USING
DELETE FROM table USING other_table WHERE table.id = other_table.id;

-- RETURNING
DELETE FROM table WHERE id = 1 RETURNING *;
```

---

4. Queries (SELECT)

Basic SELECT

```sql
SELECT col1, col2, expression AS alias
FROM table_name [alias]
WHERE condition
GROUP BY col1, col2
HAVING aggregate_condition
ORDER BY col1 [ASC|DESC] [NULLS FIRST|NULLS LAST]
LIMIT n [OFFSET m];
```

DISTINCT

```sql
SELECT DISTINCT col1, col2 FROM table;
SELECT DISTINCT ON (col1) col1, col2, col3 FROM table ORDER BY col1, col2;
```

Column Expressions

```sql
SELECT
    col + 1 AS incremented,
    UPPER(name) AS upper_name,
    CASE WHEN score > 90 THEN 'A' ELSE 'B' END AS grade,
    (SELECT COUNT(*) FROM other) AS subquery_result
FROM table;
```

WHERE Clause

Comparison operators:

```sql
WHERE col = value
WHERE col != value          -- or col <> value
WHERE col < value
WHERE col > value
WHERE col <= value
WHERE col >= value
```

Range and set:

```sql
WHERE col BETWEEN low AND high
WHERE col NOT BETWEEN low AND high
WHERE col IN (val1, val2, val3)
WHERE col NOT IN (val1, val2, val3)
WHERE col IN (SELECT ...)
```

Pattern matching:

```sql
WHERE col LIKE 'pattern'       -- % = any string, _ = any char
WHERE col NOT LIKE 'pattern'
WHERE col ILIKE 'pattern'      -- case-insensitive LIKE
WHERE col NOT ILIKE 'pattern'
```

NULL testing:

```sql
WHERE col IS NULL
WHERE col IS NOT NULL
```

Boolean logic:

```sql
WHERE cond1 AND cond2
WHERE cond1 OR cond2
WHERE NOT condition
```

Subquery predicates:

```sql
WHERE EXISTS (SELECT 1 FROM other WHERE ...)
WHERE NOT EXISTS (SELECT 1 FROM other WHERE ...)
WHERE col IN (SELECT col FROM other)
WHERE col > (SELECT AVG(col) FROM other)
```

GROUP BY

```sql
-- By column name
GROUP BY col1, col2

-- By ordinal position
GROUP BY 1, 2

-- By column alias
SELECT category AS cat, COUNT(*) FROM t GROUP BY cat

-- By computed expression
GROUP BY DATE_TRUNC('month', created_at)
```

HAVING

```sql
SELECT category, COUNT(*) AS cnt
FROM products
GROUP BY category
HAVING COUNT(*) > 10;
```

ORDER BY

```sql
ORDER BY col1 ASC, col2 DESC
ORDER BY col1 NULLS FIRST
ORDER BY col1 DESC NULLS LAST
ORDER BY 1, 2            -- by ordinal position
ORDER BY alias_name      -- by column alias
```

LIMIT / OFFSET

```sql
LIMIT 10
OFFSET 20
LIMIT 10 OFFSET 20
FETCH FIRST 10 ROWS ONLY
FETCH NEXT 5 ROWS ONLY
```

Set Operations

```sql
SELECT ... UNION SELECT ...
SELECT ... UNION ALL SELECT ...
SELECT ... INTERSECT SELECT ...
SELECT ... EXCEPT SELECT ...
```

---

5. JOINs

INNER JOIN

```sql
SELECT * FROM t1 INNER JOIN t2 ON t1.id = t2.t1_id;
SELECT * FROM t1 JOIN t2 ON t1.id = t2.t1_id;    -- INNER is default
```

LEFT / RIGHT / FULL OUTER JOIN

```sql
SELECT * FROM t1 LEFT JOIN t2 ON t1.id = t2.t1_id;
SELECT * FROM t1 RIGHT JOIN t2 ON t1.id = t2.t1_id;
SELECT * FROM t1 FULL OUTER JOIN t2 ON t1.id = t2.t1_id;
```

CROSS JOIN

```sql
SELECT * FROM t1 CROSS JOIN t2;
SELECT * FROM t1, t2;              -- implicit cross join
```

LATERAL JOIN

```sql
SELECT *
FROM t1
JOIN LATERAL (SELECT * FROM t2 WHERE t2.ref = t1.id) AS sub ON TRUE;
```

Multi-Way Joins

```sql
SELECT e.name, d.name AS dept, p.title
FROM employees e
INNER JOIN departments d ON e.dept_id = d.id
INNER JOIN projects p ON p.lead_id = e.id;
```

For 2+ inner joins, the DPccp optimizer automatically determines the optimal join order based on cardinality estimates.

---

6. Subqueries

Scalar Subquery

```sql
SELECT name, (SELECT COUNT(*) FROM orders WHERE orders.cust_id = c.id) AS order_count
FROM customers c;
```

IN Subquery

```sql
SELECT * FROM products WHERE category_id IN (SELECT id FROM categories WHERE active);
```

EXISTS Subquery

```sql
SELECT * FROM customers c
WHERE EXISTS (SELECT 1 FROM orders o WHERE o.customer_id = c.id);
```

Derived Table (Subquery in FROM)

```sql
SELECT * FROM (
    SELECT category, AVG(price) AS avg_price
    FROM products
    GROUP BY category
) AS sub
WHERE avg_price > 100;
```

---

7. Common Table Expressions (CTEs)

Simple CTE

```sql
WITH active_customers AS (
    SELECT * FROM customers WHERE status = 'active'
)
SELECT * FROM active_customers WHERE region = 'US';
```

Multiple CTEs

```sql
WITH
    cte1 AS (SELECT ...),
    cte2 AS (SELECT ... FROM cte1 ...)
SELECT * FROM cte2;
```

Recursive CTE

```sql
WITH RECURSIVE hierarchy AS (
    -- Base case
    SELECT id, name, manager_id, 1 AS depth
    FROM employees
    WHERE manager_id IS NULL

    UNION ALL

    -- Recursive case
    SELECT e.id, e.name, e.manager_id, h.depth + 1
    FROM employees e
    JOIN hierarchy h ON e.manager_id = h.id
)
SELECT * FROM hierarchy ORDER BY depth;
```

---

8. Window Functions

Syntax

```sql
function() OVER (
    [PARTITION BY col1, col2]
    [ORDER BY col1 [ASC|DESC]]
    [frame_specification]
)
```

Ranking Functions

Function	Description
`ROW_NUMBER()`	Unique sequential number within partition
`RANK()`	Rank with gaps for ties
`DENSE_RANK()`	Rank without gaps for ties
`NTILE(n)`	Distribute rows into n equal buckets
`PERCENT_RANK()`	(rank - 1) / (total_rows - 1)
`CUME_DIST()`	Cumulative distribution value

Value Functions

Function	Description
`LAG(col [, offset [, default]])`	Value from previous row
`LEAD(col [, offset [, default]])`	Value from next row
`FIRST_VALUE(col)`	First value in window frame
`LAST_VALUE(col)`	Last value in window frame
`NTH_VALUE(col, n)`	Nth value in window frame

Aggregate Window Functions

Any aggregate function can be used as a window function:

```sql
SELECT name, salary,
    SUM(salary) OVER (PARTITION BY dept ORDER BY hire_date) AS running_total,
    AVG(salary) OVER (PARTITION BY dept) AS dept_avg
FROM employees;
```

Window Frame Specification

```sql
ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW         -- default
ROWS BETWEEN UNBOUNDED PRECEDING AND UNBOUNDED FOLLOWING -- entire partition
ROWS BETWEEN n PRECEDING AND n FOLLOWING                 -- sliding window
ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING
RANGE BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW        -- value-based
```

Named Windows

```sql
SELECT
    ROW_NUMBER() OVER w AS rn,
    RANK() OVER w AS rnk
FROM employees
WINDOW w AS (PARTITION BY dept ORDER BY salary DESC);
```

9. Aggregate Functions

Basic Aggregates

Function	Description
`COUNT(*)`	Count all rows
`COUNT(col)`	Count non-NULL values
`COUNT(DISTINCT col)`	Count distinct non-NULL values
`SUM(col)`	Sum of values
`AVG(col)`	Average of values
`MIN(col)`	Minimum value
`MAX(col)`	Maximum value

Collection Aggregates

Function	Description
`STRING_AGG(col, separator)`	Concatenate with separator
`ARRAY_AGG(col)`	Collect values into array
`BOOL_AND(col)`	Logical AND of all values
`BOOL_OR(col)`	Logical OR of all values

Statistical Aggregates

Function	Description
`STDDEV_SAMP(col)`	Sample standard deviation
`STDDEV_POP(col)`	Population standard deviation
`VARIANCE(col)`	Sample variance
`VAR_SAMP(col)`	Sample variance
`VAR_POP(col)`	Population variance
`CORR(y, x)`	Correlation coefficient
`COVAR_POP(y, x)`	Population covariance
`COVAR_SAMP(y, x)`	Sample covariance

Regression Aggregates

Function	Description
`REGR_SLOPE(y, x)`	Slope of least-squares fit
`REGR_INTERCEPT(y, x)`	Y-intercept of fit
`REGR_R2(y, x)`	Coefficient of determination
`REGR_COUNT(y, x)`	Count of non-NULL pairs
`REGR_AVGX(y, x)`	Average of x values
`REGR_AVGY(y, x)`	Average of y values
`REGR_SXX(y, x)`	Sum of squares of x deviations
`REGR_SYY(y, x)`	Sum of squares of y deviations
`REGR_SXY(y, x)`	Sum of products of deviations

FILTER Clause

Apply an aggregate only to rows matching a condition:

```sql
SELECT
    COUNT(*) AS total,
    COUNT(*) FILTER (WHERE status = 'active') AS active_count,
    SUM(amount) FILTER (WHERE type = 'credit') AS credit_total
FROM transactions;
```

Ordered Aggregates

```sql
SELECT STRING_AGG(name, ', ' ORDER BY name ASC) FROM employees;
SELECT ARRAY_AGG(score ORDER BY score DESC) FROM results;
```

---

10. Built-in Scalar Functions

String Functions

Function	Description
`UPPER(str)`	Convert to uppercase
`LOWER(str)`	Convert to lowercase
`INITCAP(str)`	Capitalize first letter of each word
`LENGTH(str)`	String length in characters
`OCTET_LENGTH(str)`	String length in bytes
`SUBSTRING(str, start [, len])`	Extract substring

`POSITION(sub IN str)`	Position of substring (1-based)
`STRPOS(str, sub)`	Position of substring
`TRIM(str)`	Remove leading/trailing whitespace
`LTRIM(str [, chars])`	Remove leading characters
`RTRIM(str [, chars])`	Remove trailing characters
`BTRIM(str [, chars])`	Remove both sides
`LPAD(str, len [, fill])`	Left-pad to length
`RPAD(str, len [, fill])`	Right-pad to length
`REPEAT(str, n)`	Repeat string n times
`REVERSE(str)`	Reverse string
`REPLACE(str, from, to)`	Replace all occurrences
`CONCAT(s1, s2, ...)`	Concatenate strings
`CONCAT_WS(sep, s1, s2, ...)`	Concatenate with separator
`SPLIT_PART(str, sep, n)`	Extract nth split part
`LEFT(str, n)`	First n characters
`RIGHT(str, n)`	Last n characters
`TRANSLATE(str, from, to)`	Character-level translation
`ASCII(str)`	ASCII code of first character
`CHR(code)`	Character from ASCII code
`STARTS_WITH(str, prefix)`	Test if string starts with prefix
`OVERLAY(str, repl, start [, len])`	Replace substring
`FORMAT(fmt, ...)`	PostgreSQL format (%s, %I, %L)
`ENCODE(bytes, format)`	Encode to base64/hex/escape
`DECODE(str, format)`	Decode from base64/hex/escape
`str || str`	String concatenation operator

Regular Expression Functions

Function	Description
`REGEXP_MATCH(str, pattern)`	First match groups
`REGEXP_MATCHES(str, pattern [, flags])`	All matches
`REGEXP_REPLACE(str, pattern, repl [, flags])`	Replace matches
`REGEXP_SPLIT_TO_ARRAY(str, pattern [, flags])`	Split into array

Math Functions

Function	Description
`ABS(x)`	Absolute value
`SIGN(x)`	Sign (-1, 0, 1)
`ROUND(x [, digits])`	Round to digits
`TRUNC(x [, digits])`	Truncate to digits
`CEIL(x)` / `CEILING(x)`	Round up
`FLOOR(x)`	Round down
`POWER(base, exp)`	Exponentiation
`SQRT(x)`	Square root
`CBRT(x)`	Cube root
`EXP(x)`	e^x
`LN(x)`	Natural logarithm
`LOG(x)` / `LOG10(x)`	Base-10 logarithm
`LOG(base, x)`	Logarithm with base
`DIV(x, y)`	Integer division
`MOD(x, y)`	Modulo
`GCD(x, y)`	Greatest common divisor
`LCM(x, y)`	Least common multiple
`PI()`	Constant pi
`RANDOM()`	Random value in [0, 1)
`WIDTH_BUCKET(val, lo, hi, n)`	Assign to histogram bucket

Trigonometric Functions

`SIN(x)`, `COS(x)`, `TAN(x)`, `ASIN(x)`, `ACOS(x)`, `ATAN(x)`, `ATAN2(y, x)`, `DEGREES(x)`, `RADIANS(x)`

Date/Time Functions

Function	Description
`NOW()`	Current timestamp
`CURRENT_TIMESTAMP`	Current timestamp
`CURRENT_DATE`	Current date
`CURRENT_TIME`	Current time
`CLOCK_TIMESTAMP()`	Actual current time (not transaction time)
`EXTRACT(field FROM ts)`	Extract year/month/day/hour/minute/second/epoch/dow/doy/quarter/week
`DATE_PART(field, ts)`	Same as EXTRACT
`DATE_TRUNC(precision, ts)`	Truncate to precision
`TO_CHAR(ts, format)`	Format timestamp as string
`TO_DATE(str, format)`	Parse string to date
`TO_TIMESTAMP(str, format)`	Parse string to timestamp
`MAKE_DATE(y, m, d)`	Construct date
`MAKE_TIMESTAMP(y, m, d, h, min, sec)`	Construct timestamp
`MAKE_INTERVAL(...)`	Construct interval
`AGE(ts1 [, ts2])`	Interval between timestamps

Conditional Functions

Function	Description
`COALESCE(v1, v2, ...)`	First non-NULL value
`NULLIF(v1, v2)`	NULL if v1 = v2, else v1
`GREATEST(v1, v2, ...)`	Maximum value (ignoring NULL)
`LEAST(v1, v2, ...)`	Minimum value (ignoring NULL)

CASE Expression

```sql
CASE
    WHEN condition1 THEN result1
    WHEN condition2 THEN result2
    ELSE default_result
END

CASE expression
    WHEN value1 THEN result1
    WHEN value2 THEN result2
    ELSE default_result
END
```

Type Conversion

```sql
CAST(value AS type)
value::type                    -- PostgreSQL shorthand
TYPEOF(value)                  -- returns type name as string
TO_NUMBER(str)                 -- parse string to number
```

Sequence Functions

Function	Description
`NEXTVAL('seq_name')`	Advance and return next value
`CURRVAL('seq_name')`	Return current value
`SETVAL('seq_name', n)`	Set sequence value

Spatial Functions

Function	Description
`POINT(x, y)`	Construct a point from longitude and latitude
`ST_Distance(p1, p2)`	Haversine great-circle distance in meters
`ST_Within(p1, p2, dist)`	True if distance <= dist meters
`ST_DWithin(p1, p2, dist)`	Alias for ST_Within

```sql
-- Compute distance between two points
SELECT ST_Distance(POINT(-73.99, 40.73), POINT(-73.98, 40.76)) AS dist_m;

-- Boolean distance predicate in WHERE
SELECT name FROM restaurants
WHERE ST_DWithin(location, POINT(-73.9903, 40.7359), 1500);
```

Miscellaneous

Function	Description
`GEN_RANDOM_UUID()`	Generate random UUID v4
`ISFINITE(value)`	Check if value is finite

---

11. JSON/JSONB Operators and Functions

Access Operators

Operator	Description	Result Type
`col -> key`	Get JSON field by key	JSON
`col ->> key`	Get JSON field by key	TEXT
`col -> index`	Get array element by index	JSON
`col ->> index`	Get array element by index	TEXT
`col #> '{k1,k2}'`	Get nested field by path	JSON
`col #>> '{k1,k2}'`	Get nested field by path	TEXT

Containment Operators

Operator	Description
`col @> other`	Left contains right
`col <@ other`	Left contained by right
`col ? key`	Has key
`col ?| array`	Has any of these keys
`col ?& array`	Has all of these keys

Construction Functions

Function	Description
`json_build_object(k1, v1, k2, v2, ...)`	Build JSON object
`json_build_array(v1, v2, ...)`	Build JSON array
`to_json(value)`	Convert to JSON
`to_jsonb(value)`	Convert to JSONB
`row_to_json(row)`	Convert row to JSON object

Inspection Functions

Function	Description
`json_typeof(col)`	Type name: object, array, number, string, boolean, null
`json_array_length(col)`	Array length
`json_extract_path(col, k1, k2, ...)`	Extract by path (JSON result)
`json_extract_path_text(col, k1, k2, ...)`	Extract by path (TEXT result)

Modification Functions

Function	Description
`jsonb_set(target, path, value [, create])`	Set value at path
`jsonb_insert(target, path, value)`	Insert value at path
`jsonb_strip_nulls(col)`	Remove NULL values

Set-Returning Functions

Function	Description
`json_each(col)`	Key-value pairs
`json_each_text(col)`	Key-value pairs (text values)
`json_array_elements(col)`	Array elements
`json_array_elements_text(col)`	Array elements as text
`json_object_keys(col)`	Object keys

All functions above have `jsonb_` equivalents.

---

12. Array Functions

Function	Description
`ARRAY_LENGTH(arr, dim)`	Length of array dimension
`ARRAY_UPPER(arr, dim)`	Upper bound of dimension
`ARRAY_LOWER(arr, dim)`	Lower bound of dimension
`ARRAY_CAT(arr1, arr2)`	Concatenate arrays
`ARRAY_APPEND(arr, elem)`	Append element
`ARRAY_REMOVE(arr, elem)`	Remove all occurrences
`CARDINALITY(arr)`	Total element count

---

13. Cross-Paradigm Search Functions

These functions operate in the WHERE clause and produce relevance scores accessible via the `_score` column.

Full-Text Search

```sql
-- BM25 full-text search
SELECT title, _score FROM articles
WHERE text_match(title, 'transformer attention')
ORDER BY _score DESC;

-- Bayesian BM25 (calibrated probability in [0,1])
SELECT title, _score FROM articles
WHERE bayesian_match(title, 'transformer attention')
ORDER BY _score DESC;
```

Vector Similarity Search

```sql
-- K-nearest neighbors
SELECT title, _score FROM papers
WHERE knn_match(embedding, ARRAY[0.1, 0.2, ...], 10)
ORDER BY _score DESC;

-- With parameter binding
SELECT title, _score FROM papers
WHERE knn_match(embedding, $1, 5);
```

Graph Traversal

```sql
-- Vertices reachable within max_hops
SELECT * FROM traverse(start_id, 'edge_label', max_hops);

-- Regular path query (Kleene star, alternation, concatenation)
SELECT * FROM rpq('manages*', 1);
SELECT * FROM rpq('manages/has_skill', 2);

-- Reachability predicate in WHERE
SELECT * FROM employees
WHERE traverse_match(1, 'manages', 3);
```

Bounded RPQ

```sql
-- Bounded repetition: paths of 2-3 hops
SELECT * FROM rpq('knows{2,3}', 1);
```

Graph Centrality

```sql
-- PageRank as table source (FROM clause)
SELECT _doc_id, title, _score FROM pagerank() ORDER BY _score DESC;
SELECT _doc_id, _score FROM pagerank(0.95) ORDER BY _score DESC;  -- custom damping
SELECT _doc_id, _score FROM pagerank('social') ORDER BY _score DESC;  -- named graph (direct name)

-- HITS hub/authority scoring
SELECT _doc_id, title, _score FROM hits() ORDER BY _score DESC;

-- Betweenness centrality
SELECT _doc_id, title, _score FROM betweenness() ORDER BY _score DESC;

-- PageRank as WHERE signal (combined with relational filter)
SELECT title, year, _score FROM papers WHERE pagerank() AND year >= 2020 ORDER BY _score DESC;

-- PageRank with named graph in WHERE signal
SELECT title, _score FROM papers WHERE pagerank('social') ORDER BY _score DESC;
SELECT title, _score FROM papers WHERE pagerank(0.90, 50, 1e-5, 'social') ORDER BY _score DESC;

-- Centrality as fusion signal (with optional named graph)
SELECT title, _score FROM papers
WHERE fuse_log_odds(text_match(title, 'attention'), pagerank('social'))
ORDER BY _score DESC;
```

Weighted Path Query

```sql
-- Sum of edge weights along path
SELECT title, _score FROM papers
WHERE weighted_rpq('cites/cites', 7, 'weight', 'sum')
ORDER BY _score DESC;

-- Max aggregate with threshold predicate
SELECT title, _score FROM papers
WHERE weighted_rpq('cites/cites', 7, 'weight', 'max', 6.0)
ORDER BY _score DESC;
```

Graph Mutation (SQL)

```sql
-- Add graph vertex to a table's graph store
SELECT * FROM graph_add_vertex(1, 'person', 'employees');
SELECT * FROM graph_add_vertex(2, 'person', 'employees',
'name=Alice,age=30');

-- Add graph edge to a table's graph store
SELECT * FROM graph_add_edge(1, 1, 2, 'knows', 'employees');
SELECT * FROM graph_add_edge(2, 1, 2, 'knows', 'employees',
'weight=0.8,since=2020');
```

Progressive Fusion

```sql
-- Cascading multi-stage WAND fusion
SELECT title, _score FROM papers
WHERE progressive_fusion(
    text_match(title, 'attention'),
    traverse_match(1, 'cites', 2),
    5,
    pagerank(),
    3
) ORDER BY _score DESC;

-- With gating
WHERE progressive_fusion(sig1, sig2, 5, sig3, 3, 'relu')
```

Spatial Search

```sql
-- All restaurants within 2km of a point (R*Tree-accelerated)
SELECT name, cuisine FROM restaurants
WHERE spatial_within(location, POINT(-73.9973, 40.7308), 2000)
ORDER BY name;

-- With computed distance
SELECT name,
       ROUND(ST_Distance(location, POINT(-73.9857, 40.7484)), 0) AS dist_m
FROM restaurants
WHERE spatial_within(location, POINT(-73.9857, 40.7484), 5000)
ORDER BY dist_m;

```

```
-- Parameter binding for center and radius
SELECT name FROM restaurants
WHERE spatial_within(location, $1, $2)
ORDER BY name;
-- params: [[-73.9973, 40.7308], 2000]
```

`spatial_within(field, POINT(x, y), distance_meters)` returns all points within the given radius, scored by proximity (`1.0 - dist/max_dist`). Uses the R*Tree index if one exists on the column, otherwise falls back to brute-force Haversine scan.

Vector Exclusion

```
-- Find similar items while excluding a negative query
SELECT * FROM products
WHERE vector_exclude(embedding, positive_vec, negative_vec, k, threshold);
```

---

14. Probabilistic Fusion

Combine multiple relevance signals into a single fused score. All signals are calibrated to probabilities before fusion.

Log-Odds Fusion

```
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5)
)
ORDER BY _score DESC;
```

Log-Odds Fusion with Gating

```
-- ReLU gating: suppress weak negative evidence
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5),
    'relu'
) ORDER BY _score DESC;

-- Swish gating: smooth approximation to ReLU
SELECT title, _score FROM papers
WHERE fuse_log_odds(
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5),
    'swish'
) ORDER BY _score DESC;

-- Alpha parameter + gating combined
SELECT title, _score FROM papers
WHERE fuse_log_odds(
```

```
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5),
    0.8,
    'relu'
) ORDER BY _score DESC;
```

Valid gating values: `'relu'`, `'swish'`. Default is no gating. The gating parameter is always the last argument (after optional alpha).

Probabilistic AND

```
-- Both signals must indicate relevance (product rule)
SELECT title, _score FROM papers
WHERE fuse_prob_and(
    text_match(title, 'attention'),
    knn_match(embedding, $1, 5)
);
```

Probabilistic OR

```
-- Either signal indicates relevance (inclusion-exclusion)
SELECT title, _score FROM papers
WHERE fuse_prob_or(
    bayesian_match(title, 'attention'),
    traverse_match(42, 'cited_by', 2)
);
```

Probabilistic NOT

```
WHERE fuse_prob_not(knn_match(embedding, $1, 5))
```

Spatial + Fusion

`spatial_within()` can be used as a fusion signal alongside text, vector, and graph signals:

```
SELECT name, _score FROM restaurants
WHERE fuse_log_odds(
    text_match(description, 'pizza'),
    spatial_within(location, POINT(-73.9969, 40.7306), 3000)
) ORDER BY _score DESC;
```

---

15. Cypher / Graph Integration

Named Graph Management

Named graphs partition the graph store into isolated namespaces. Each named graph has its own adjacency indexes (per-graph `_GraphPartition`) while vertices and edges are stored globally. Named graphs are persisted via `_graph_catalog` and `_graph_membership` SQLite tables.

```sql
-- Create and drop named graphs
SELECT * FROM create_graph('social');
SELECT * FROM drop_graph('social');
```

All graph functions accept direct graph names without the `graph:` prefix (backward compatible):

```sql
-- Traverse, RPQ, and centrality with named graphs
SELECT * FROM traverse(1, 'knows', 2, 'social');
SELECT * FROM rpq('knows/works_with', 1, 'social');
SELECT * FROM temporal_traverse(1, 'knows', 2, 150, 'net');
SELECT * FROM pagerank(0.85, 'social');
SELECT * FROM hits(20, 'social');
SELECT * FROM betweenness('social');
```

Cypher Queries in SQL

```sql
-- Query the default graph
SELECT * FROM cypher('
    MATCH (p:Person)-[:KNOWS]->(f:Person)
    WHERE p.name = ''Alice''
    RETURN f.name AS friend, f.age
');

-- Query a named graph
SELECT * FROM cypher('
    MATCH (n:Person)
    RETURN n.name, n.age
    ORDER BY n.age DESC
    LIMIT 10
', 'social');
```

Supported Cypher Clauses

Clause	Description
`MATCH`	Pattern matching
`OPTIONAL MATCH`	Pattern matching (NULL for no match)
`WHERE`	Filter conditions
`RETURN`	Project results
`WITH`	Intermediate projection (pipelining)
`CREATE`	Create vertices and edges
`SET`	Update properties
`DELETE`	Delete vertices and edges
`MERGE`	Match or create
`UNWIND`	Expand list to rows
`ORDER BY`	Sort results
`LIMIT`	Limit result count

Cypher Pattern Syntax

```
(n)                     -- any vertex
(n:Person)              -- vertex with label
(n:Person {name: 'Alice'})  -- vertex with properties
(n)-[r]->(m)            -- directed edge
(n)<-[r]-(m)            -- reverse direction
(n)-[r]-(m)             -- undirected edge
(n)-[r:KNOWS]->(m)      -- edge with label
(n)-[r:KNOWS*1..3]->(m)  -- variable-length path
(n)-[r:KNOWS*]->(m)     -- unbounded variable-length
```

Cypher Functions

`id(n)`, `label(n)`, `properties(n)`, `type(r)`, `exists(expr)`, `keys(n)`, `relationships(path)`

---

16. Table-Returning Functions

GENERATE_SERIES

```sql
-- Integer series
SELECT * FROM generate_series(1, 10) AS t(n);
SELECT * FROM generate_series(0, 100, 10) AS t(n);

-- Timestamp series
SELECT * FROM generate_series(
    '2024-01-01'::timestamp,
    '2024-12-31'::timestamp,
    '1 month'::interval
) AS t(d);
```

REGEXP_SPLIT_TO_TABLE

```sql
SELECT * FROM regexp_split_to_table('a,b,c,d', ',') AS t(val);
```

---

17. Information Schema

Tables Metadata

```sql
SELECT table_name, table_type
FROM information_schema.tables
WHERE table_schema = 'public';
```

Columns: `table_catalog`, `table_schema`, `table_name`, `table_type`, `is_insertable_into`, `is_typed`, `commit_action`

Column Metadata

```sql
SELECT column_name, data_type, is_nullable, column_default
FROM information_schema.columns
WHERE table_name = 'employees';
```

Columns: `table_catalog`, `table_schema`, `table_name`, `column_name`, `ordinal_position`, `column_default`, `is_nullable`, `data_type`, `character_maximum_length`, `numeric_precision`, `numeric_scale`

---

18. EXPLAIN

```sql
-- Show query plan
EXPLAIN SELECT * FROM t WHERE id > 10;

-- Show plan with execution statistics
EXPLAIN ANALYZE SELECT * FROM t WHERE id > 10;

-- Verbose output
EXPLAIN (ANALYZE true, VERBOSE true) SELECT * FROM t JOIN t2 ON t.id =
t2.tid;
```

19. ANALYZE

Collect column statistics for the query optimizer:

```sql
-- Analyze a specific table
ANALYZE table_name;

-- Analyze all tables
ANALYZE;
```

Statistics collected: distinct count, NULL count, min/max values, equi-depth histograms, most common values and frequencies.

20. Transactions

```sql
BEGIN;
-- ... statements ...
COMMIT;

BEGIN;
-- ... statements ...
ROLLBACK;

-- Savepoints
BEGIN;
INSERT INTO t VALUES (1);
SAVEPOINT sp1;
INSERT INTO t VALUES (2);
ROLLBACK TO SAVEPOINT sp1;  -- undoes second insert
RELEASE SAVEPOINT sp1;
COMMIT;
```

21. Prepared Statements

```sql
PREPARE my_query (INTEGER, TEXT) AS
    SELECT * FROM users WHERE id = $1 AND name = $2;

EXECUTE my_query (42, 'Alice');

DEALLOCATE my_query;
```

22. Operators Reference

Comparison

`=`, `!=`, `<>`, `<`, `>`, `<=`, `>=`

Arithmetic

`+`, `-`, `*`, `/`, `%`

String

`||` (concatenation)

Pattern Matching

`LIKE`, `NOT LIKE`, `ILIKE`, `NOT ILIKE`, `SIMILAR TO`

Logical

`AND`, `OR`, `NOT`

Range / Set

`BETWEEN ... AND ...`, `IN (...)`, `IS NULL`, `IS NOT NULL`

JSON

`->`, `->>`, `#>`, `#>>`, `@>`, `<@`, `?`, `?|`, `?&`